

Test Task: Full-Stack Developer

GOAL

Build a small, well-structured full-stack application to demonstrate your skills in C#, .NET REST APIs, React, and modern frontend practices. We will primarily review your **code quality, architecture, and reasoning**; a fully polished product is not required.

TIMEBOX

This task is intentionally scoped for **4–8 hours of focused work**. Prioritize the core requirements. It's acceptable to stub/mock secondary parts (e.g., notification transport, image storage) and document trade-offs in the README.

SCENARIO

Implement a minimal "Shop" with: authentication, product listing with infinite scroll, and a shopping cart that persists. After a successful login, trigger a simple notification workflow on the backend (e.g., enqueue/send a mock notification) and expose its status to the client.

Functional Requirements

- **User Authentication:**
 - Login with username & password.
 - On success, return basic user profile info
 - On failure, return a clear error (no stack traces).
- **Products:**
 - Home page shows a list of products with **infinite scroll** (load more on scroll).
 - Each product: name, price, image/placeholder, and a short description (your call).
- **Cart:**
 - Add/remove items; update quantity.
 - Cart total updates accordingly.
 - **Cart persists** across page refreshes (localStorage or API session—your choice; justify in README).
- **Post-Login Notification**
 - After successful login, the backend triggers a **notification** (mock is fine: log entry, in-memory queue, or a simple table record) and exposes an endpoint to fetch its latest status.
- **Error Handling**
 - Frontend: user-friendly error messages and fallback UI.
 - Backend: proper status codes, validation, and structured error responses

Technical Requirements:

Backend (.NET/C#)

- .NET 8+ Web API.
- Use MsSQL or MySQL for auth & products.
- Clean architecture or Isayered approach (Controllers → Services → Repos; DTOs; mapping).
- Consider designing your REST services as if there will be many services in the future. If useful, factor a small reusable library/shared package for cross-cutting concerns (e.g., error handling/result types, paging, notifications).
- Seed sample data via migration or script.
- Add minimal unit tests for at least one domain/service component.

Frontend (React)

- React 18+ with React Router.
- UI with MUI (Material UI).
- State management: your choice (Context, Zustand, Redux, or RxJS). If you use RxJS, show how "services & observables" shape data flow.
- Implement infinite scroll on the products page.
- Persist cart across refreshes.
- TypeScript preferred.

Evaluation Metrics

Your submission will be evaluated on the following:

- **Code Quality:** This includes readability, organization, and adherence to standard coding conventions.
- **Architecture and Design:** How well you utilize object-oriented programming principles and best practices in your design and architecture.
- **Error Management:** Your ability to handle potential errors and edge cases effectively.
- **Frontend Practices:** Component structure, hooks, data fetching, UX polish.

Submission Instructions

- **GitHub Repository** with two folders (or a monorepo): `backend/` and `frontend/`.
- **README.md** at the root describing:
 - How to run backend & frontend locally (commands, env vars, DB setup, seed data).
 - Architecture decisions & trade-offs (why this state management, paging approach, etc.).
 - What you would do next with more time.
- Optional: Postman/Thunder tests or a small OpenAPI spec.

Ensure your submission is well-documented and organized to facilitate easy review and understanding of your work.

NOTE

Feel free to use any additional libraries or tools that you think might be useful for this project. You may also add unit tests if you wish to showcase your testing skills. Remember, the goal of this task is to assess your proficiency in C# development.

We focus on reading code and reviewing architecture. A fully deployed demo is **not** required.

Good luck!

P.S. AGAIN ... It is not important for us to see this app working. We are going to read a code and see the architecture.